

VY LAB 3. Hafta Linked List Örnekler

Bağlı listede işaretçiler için kısa not

`Node* head = NULL;` ifadesinde `head` bir düğüm değildir; ilk düğümün adresini tutan işaretçidir. Liste boşken göstereceği düğüm olmadığı için `NULL` değerini alır.

Örneğin `head → [10] → [20] → [30]` listesinde `head` ilk düğümü gösterir; 10 ise o düğümün data değeridir. Başa [5] eklendiğinde `head → [5] → [10] → [20] → [30]` olur. `head` aynı değişkendir, yalnızca tuttuğu adres değişmiştir.

`Node*` bir düğümün adresini tutar. `&head`, `head` değişkeninin kendi adresidir; bu adresin türü `Node**` olur. `addOrdered(&head, 5)` çağrısında `&head` ile fonksiyona `head` değişkenine erişme olanağı, 5 ile yeni düğüme yazılacak veri gönderilir.

Fonksiyondaki parametrenin adı `headAdresi` olsaydı `Node** headAdresi` şeklinde tanımlanırdı. `*headAdresi = newNode;` satırı, “`head` değişkeninin içine yeni düğümün adresini yaz” demektir. Böylece ilk düğüm değişse bile `main` içindeki `head` doğru düğümü gösterir. Yalnızca listeyi okuyan `printList(Node* head)` gibi fonksiyonlarda `Node*` yeterlidir.

Örnek-1:

Yapılacaklar:

Bir **tek yönlü bağlı liste** kullanarak aşağıdaki fonksiyonları C dilinde yazınız:

- `void addOrdered(Node** head, int value);`

Bağlı listeye **sıralı şekilde** yeni bir düğüm ekler. Liste boşsa yeni düğüm baş olur. Bu fonksiyonun sıralı sonuç üretmesi için mevcut liste sıralı olmalıdır; fonksiyon önceki düğümleri yeniden sıralamaz.

Örneğin: Boş listeye sırasıyla 23, 11, 5, 9, 6, 4, 12, 24 sayıları bu fonksiyonla eklendiğinde listenin içeriği `4 → 5 → 6 → 9 → 11 → 12 → 23 → 24` şeklinde olmalıdır.

- `void removeNode(Node** head, int value);`

Bağlı listede verilen değere sahip **ilk düğümü** siler.

- `int count(Node* head);`

Listedeki düğüm sayısını döndürür.

- `void printList(Node* head);`

Listenin elemanlarını **baştan sona** ekrana yazdırır.

- `void clear(Node** head);`

Listedeki tüm düğümleri serbest bırakır (free eder) ve listeyi boşaltır.

Örnek-2:

Bir **tek yönlü bağlı liste** üzerinde pozisyona göre **ekleme** ve **silme** işlemlerini gerçekleştiren C programını yazınız.

Aşağıdaki fonksiyonları oluşturmanız beklenmektedir:

- `void insertAt(Node** head, int value, int position);`

Verilen `value` değerini, bağlı listenin `position` (indis) numaralı pozisyonuna ekler.

0 veya negatif pozisyona ekleme yapıldığında, yeni düğüm listenin başına eklenmiş olur.

Eğer verilen pozisyon, listedeki eleman sayısından büyükse, yeni düğüm **listenin sonuna** eklenir.

- `void deleteAt(Node** head, int position);`

Verilen pozisyondaki düğümü listeden siler.

0. pozisyondaki düğüm silinirse, baş düğüm bir sonraki düğümle güncellenir.

Geçersiz bir pozisyon girilirse işlem yapılmaz.

- `void printList(Node* head);`

Listenin elemanlarını baştan sona yazdırır.

- `void clear(Node** head);`

Listedeki tüm düğümleri serbest bırakır (free eder) ve listeyi boşaltır.

Örnek-3:

Bir **tek yönlü bağlı liste** içindeki **ortadaki düğümün** değerini bulan bir C programı yazınız.

Aşağıdaki fonksiyonları oluşturmanız beklenmektedir:

- `Node* findMiddle(Node* head);`

Bağlı listedeki **ortadaki düğümü** döndürür.

Eğer listedeki eleman sayısı **tek** ise, ortadaki düğüm döndürülür.

Eğer eleman sayısı **çift** ise, iki ortadan **ikincisini** döndürür.

Boş liste durumunda `NULL` döndürülür.

Bu işlem için **slow** ve **fast** göstericileri (iki pointer) kullanabilirsiniz:

- `slow` → her adımda bir düğüm ilerler
- `fast` → her adımda iki düğüm ilerler
fast sona ulaştığında slow ortadadır.

- `void printList(Node* head);`

Listenin elemanlarını baştan sona yazdırır.

- `void clear(Node** head);`

Listedeki tüm düğümleri serbest bırakır (free eder) ve listeyi boşaltır.